

Collection Framework

Collection framework is use for store and manipulation the group of objects.

Iterable Interface

It is root interface of collection classes.

It contains only one abstract method - `Iterator<T> iterator()`.

Iterator Interface

It will have functionality it iterat the element in Forword direction only.

It have `next()`, `hasNest()`, `remove()` and `forEachRemaining()`.

Collection Interface

It is child interface of iterable interface.

It is used to represent the group of individual objects as a single unit.

It has some method like `add()`, `addAll()`, `remove()`, `removeAll()` and so on.

Collections Class

It is utility class which has some method that are used to operate on collections.

It contains only Static method.

Like `Collections.max(List)`, `Collections.min(List)`.

List Interface

- **ArrayList and LinkedList** are both the contains duplicate elements.
- Both are maintain insertion order.
- Both are non-synchronized.
- Now ArrayList uses Dynamic array to store the elments.
- And LinkedList uses Doubly LinkedList internally to store the elements.
- ArrayList is slower than Linked list because of lots of shifting is needed.
- LinkedList is faster than ArrayList.
- We need to use ArrayList when we want to read faster.
- And we need to use LinkedList when we want to addition and remove the elements.
- ArrayList – Read.
- LinkedList – Manipulation

How to write CustomeArrayList which is not accept duplicate element.

We have to extends ArrayList on class.

Override the `add()` method.

Write a logic in this `add()` method.

```
public class CustomArrayList extends ArrayList {  
  
    @Override  
    public boolean add(Object o) {  
        if(this.contains(o)){  
            return true;  
        }else{  
            return super.add(o);  
        }  
    }  
  
    public static void main(String[] args) {  
  
        CustomArrayList list1=new CustomArrayList();  
        list1.add(1);  
        list1.add(1);  
        list1.add(1);  
        list1.add(1);  
    }  
}
```

ArrayDeque

It contain unique elements.

HashMap	Hashtable
1) HashMap is non synchronized . It is not-thread safe and can't be shared between many threads without proper synchronization code.	Hashtable is synchronized . It is thread-safe and can be shared with many threads.
2) HashMap allows one null key and multiple null values .	Hashtable doesn't allow any null key or value .
3) HashMap is a new class introduced in JDK 1.2 .	Hashtable is a legacy class .
4) HashMap is fast .	Hashtable is slow .
5) We can make the HashMap as synchronized by calling this code Map m = Collections.synchronizedMap(hashMap);	Hashtable is internally synchronized and can't be unsynchronized.
6) HashMap is traversed by Iterator .	Hashtable is traversed by Enumerator and Iterator .
7) Iterator in HashMap is fail-fast .	Enumerator in Hashtable is not fail-fast .
8) HashMap inherits AbstractMap class.	Hashtable inherits Dictionary class.

Comparable – Single sorting – compareTo()

Comparator – Multiple Sorting – compare()

Comparable	Comparator
1) Comparable provides a single sorting sequence . In other words, we can sort the collection on the basis of a single element such as id, name, and price.	The Comparator provides multiple sorting sequences . In other words, we can sort the collection on the basis of multiple elements such as id, name, and price etc.
2) Comparable affects the original class , i.e., the actual class is modified.	Comparator doesn't affect the original class , i.e., the actual class is not modified.
3) Comparable provides compareTo() method to sort elements.	Comparator provides compare() method to sort elements.
4) Comparable is present in java.lang package.	A Comparator is present in the java.util package.
5) We can sort the list elements of Comparable type by Collections.sort(List) method.	We can sort the list elements of Comparator type by Collections.sort(List, Comparator) method.

Equals and hashCode Contract

First both are present in Object Class.

If we override the equals() method, then it is mandatory to override the hashCode() method.

Now the contract is after we override this equals method then If two different object of equals() method will return true then this two object must have same hashCode.

For example

We have student class which having two fields id and name.

Both student object having same id but different name.

Now if we override the method then we will compare with id and it will return true then we must override the hashCode() method.

MAP Interface

It contains key – value pair
put(), putAll()

Map.Entry Interface

It sub interface of map.

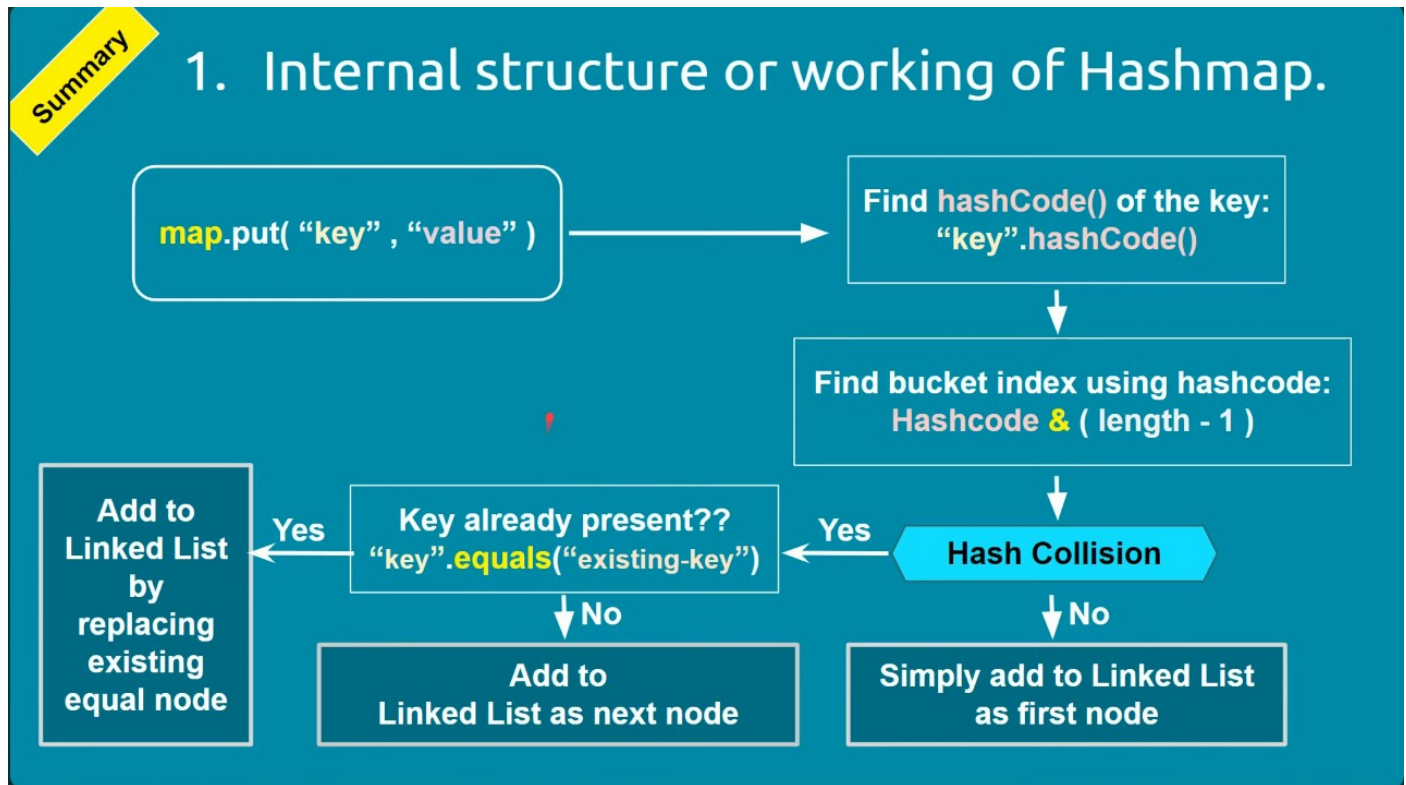
It will return collection view of the map.

It provide a method to get key and value like getKey(), getValue(), setValue(),

```
Map<Integer,String> map=new HashMap<>();  
map.put(100,"Amit");  
map.put(101,"Vijay");  
map.put(102,"Rahul");  
//Elements can traverse in any order  
for(Map.Entry m:map.entrySet()){  
    System.out.println(m.getKey()+" "+m.getValue());  
}
```

HashMap (Key K, Value V)

Internal Working hashMap



Internally use Hashing Technique.

equals():

It checks the equality of two objects.

It compares the Key, whether they are equal or not.

It is a method of the Object class. It can be overridden.

If you override the equals() method, then it is mandatory to override the hashCode() method.

hashCode():

This is the method of the object class.

It returns the memory reference of the object in integer form.

The value received from the method is used as the bucket number.

The bucket number is the address of the element inside the map. Hash code of null Key is 0.

Buckets:

Array of the node is called buckets.

Each node has a data structure like a LinkedList.

More than one node can share the same bucket.

It may be different in capacity.

- When we create Map Object then it will create Bucket.
- Initial capacity of map is 16 then Default Bucket structure is 16 means 0 – 15.
- This all bucket is consider as a LinedList.
- This LinkedList contains Node.
- Node has a **Hash Key Value next**
- Now when call put(K, V) method it will calculate hashCode using key and then calculate bucket index value using hashcode & (length - 1) for this hasValue.

- Now it will put this **hashValue Key Value next** in calculated bucket index.
- Now next is null because no new node is insert in bucket yet.
- If next map object put method is call and this object calculated index of bucket is same as present one then hashing collision happens.
- Collision means if same bucket having multiple nodes.
- Now when Collision happens it will call internally equals() method.
- It will check and compare present node key and current node key is same or not using equals() method.
- If it is different then immediately it will store this current node in same bucket.
- Now present reference node next value having this new node reference.
- If Key as Null then it will go with zeroth bucket.

What is enhancement of HashMap in Java 8?

This HashMap is use bucket as a LinkedList but in certain threshold it will convert as balance tree mechanism. In Java 8 they use Balance tree mechanism instead of LinkedList in certain threshold.

What is Fail-Fast and Fail-Safe?

Fail-fast is like throw ConcurrentModificationException while the two threads are try to modifying simultaneously then it will throw the exception.

Fail-safe is like working on clone of its collection and it can't throw any exception.

Concurrent Collection

CopyOnWriteArrayList – same as ArrayList but its thread safe.

CopyOnWriteArraySet – same as set but thread safe and it internally uses CopyOnWriteArrayList for operations.

Why we need of concurrent collection in Java?

If we try over traditional collection object operation like simultaneously run thread and try to modification on existing structure then it will throw concurrentModificationException.

Now we need operation in thread safe manner then we need to us concurrent collection.

Why performance of concurrentHashMap is better than HashTable and SynchronizedMap?

In traditional collection if any operation performs then lock is acquired on complete collection so only single thread can capture.

But in concurrency collection the lock will acquired only in bucket level.

What is Concurrency level in ConcurrentHashMap?

Concurrency level defines how many threads are allowed to get Lock.

By default, ConcurrentHashMap has 16 buckets and also concurrency level has 16 values.

If concurrency level is 8 and bucket is 16 then each segment get lock on two buckets.

If thread1 and thread2 will working on different segment

Now if we want update segment of t1 with t2 it will not allow

But read operation will work properly.

Why null not allowed in ConcurrentHashMap while allowed in hashMap?

Can concurrent modification exception occur only in multithreaded environment?

No.

What is the difference between HashMap and ConcurrentHashMap ?

HashMap

HashMap is thread unsafe

Performance is high

HashMap can contain null key and value both

It is fail fast, if one thread is iterating map and other thread

tries to do structural change then

we get ConcurrentModificationException.

ConcurrentHashMap

ConcurrentHashMap is thread safe

Performance is low

ConcurrentHashMap can not contain null as a key/value pair.

ConcurrentHashMap is fail safe , if one thread is iterating map then other can update same map.

What is the difference between ArrayList and CopyOnWriteArrayList?

ArrayList

ArrayList is thread unsafe

Performance is high

It is fail fast, if one thread is iterating ArrayList and other thread tries to do structural change then we get ConcurrentModificationException.

Iterator can remove element

CopyOnWriteArrayList

CopyOnWriteArrayList is thread safe

Performance is low

CopyOnWriteArrayList is fail safe , if one thread is iterating List then other tries to update List then new clone will be created and later JVM will synchronize the clone and actual copy.

if Iterator tries to remove element then UnsupportedOperationException exception is thrown.